

Summary

This video explains the technical synthesis of the conversion of Finite Automata (FA) into Regular Expressions (RE), with primary emphasis on Arden's Lemma. Although two principal techniques exist—Arden's Lemma and the State Elimination Method—Arden's Lemma provides a rigorous algebraic framework for solving systems of state equations.

The core theorem states that for an equation of the form:

$$R = Q + RP$$

there exists a unique solution:

$$R = QP^*$$

provided that P does not contain the null string (ϵ). This algebraic method is especially powerful in competitive examinations such as GATE, where recognizing structural similarities in state equations can enable rapid equivalence detection without exhaustive string testing.

1. Theoretical Foundation: Arden's Lemma

Arden's Lemma is a rule for solving linear equations over regular expressions. It systematically determines the language accepted by each state in a finite automaton.

Fundamental Theorem

Let P and Q be regular expressions over alphabet Σ . If P does not contain ϵ , then the equation:

$$R = Q + RP$$

has the unique solution:

$$R = QP^*$$

Proof Sketch and Logical Derivation

Start with:

$$R = Q + RP$$

Substitute R recursively:

$$\begin{aligned} R &= Q + (Q + RP)P \\ &= Q + QP + RP^2 \end{aligned}$$

Repeat substitution:

$$R = Q + QP + QP^2 + RP^3$$

Continuing this expansion yields:

$$R = Q + QP + QP^2 + QP^3 + \dots$$

Factor Q:

$$R = Q(\epsilon + P + P^2 + P^3 + \dots)$$

Recognizing the Kleene star expansion:

$$\epsilon + P + P^2 + P^3 + \dots = P^*$$

Therefore:

$$R = QP^*$$

Uniqueness Condition

If P contains ϵ , then multiple solutions may exist, violating uniqueness. Therefore, the lemma applies only when P does not include ϵ .

2. Procedural Application to Finite Automata

The conversion process follows a systematic algebraic framework.

Step 1: Equation Generation

Write one equation per state. Each equation expresses the language of that state in terms of transitions into it.

Step 2: Incoming Transition Rule

If there is a transition from state A to state B on symbol 0, then include term A0 in the equation for B.

Step 3: Initial State Rule

The equation of the initial state must include:

- ϵ

This represents reachability without consuming input.

Step 4: Solve Using Arden's Lemma

Rearrange each equation into the form:

$$R = Q + RP$$

Then apply:

$$R = QP^*$$

Step 5: Final State Rule

If $F_1, F_2, \dots, F_n$ are final states, the overall regular expression is:

$$F_1 + F_2 + \dots + F_n$$

3. Analysis of Complex Conversions

Integer Divisibility by Three (Binary Automaton)

Consider an automaton representing binary numbers divisible by 3, with states:

q_0 (remainder 0)

q_1 (remainder 1)

q_2 (remainder 2)

State equations:

$$q_0 = q_00 + q_11 + \epsilon$$

$$q_1 = q_01 + q_20$$

$$q_2 = q_10 + q_21$$

By systematically applying substitution and Arden's Lemma, the final expression for q_0 (the accepting state) becomes:

$$0^*1(01^*0)^*10^*$$

Different algebraic elimination orders produce syntactically different but equivalent expressions.

Even Number of Zeros and Ones

For a four-state automaton tracking parity of 0s and 1s:

States represent combinations of (even/odd zeros, even/odd ones).

The system requires repeated isolation of variables and nested applications of Arden's Lemma. Although algebraically intensive, it demonstrates that even parity-tracking automata can be fully reduced to regular expressions through systematic manipulation.

Multiple Equivalent Regular Expressions

The same automaton can yield different regular expressions depending on elimination order.

Example:

Language: Strings ending with 'b'

Possible REs:

$(a+b)^*b$

More complex algebraic forms derived via state equations

All are equivalent in language, differing only syntactically.

4. Strategic Comparison: Arden's Lemma vs State Elimination

Feature	Arden's Lemma	State Elimination Method
Approach	Algebraic equation solving	Graphical state removal
Nature	Symbolic and substitution-based	Visual and structural
Speed (Small FA)	Efficient	Efficient
Speed (Large FA)	Can become algebraically heavy	Generally faster
Best Use Case	Structural comparison and equivalence	General FA-to-RE conversion

Drawback of Arden's Lemma

To derive the expression for the final state, it is often necessary to solve equations for all other states first. This sequential dependency increases algebraic complexity in large automata.

Competitive Advantage: GATE 2005 Case

Two final states y and z in an NFA had equations:

$$y = x0 + y0 + z1$$

$$z = x0 + y0 + z1$$

Since both equations are identical, it immediately follows that:

$$L(y) = L(z)$$

This insight eliminates the need for full elimination or string testing. Arden's Lemma enables structural equivalence detection directly from equations.

